Campus Sports Facility Reservation System – Final Plan

BMIT2023 Web and Mobile Systems group assignment. Three members build an online reservation system for TARUMT sports facilities, such as courts and the Arena gym. The tutor has approved the title and the team size.

1. Overview

Members book one-hour slots at campus facilities. Students book free after a one-time approval, and Public members pay online. Admins run facilities, slots, members and bookings, and one seeded Super Admin manages admin accounts.

Item	Decision
Team	Member 1, Member 2, Member 3 (replace with real names)
Framework	.NET 10, ASP.NET Core MVC, C#
Data	EF Core 10.x code-first with data annotations, SQL Server Express file-based `.mdf`
Security	Manual cookie authentication, built-in `PasswordHasher`, no ASP.NET Core Identity
Client libraries	jQuery 3.7.1 (not 4.x), jQuery Validate, Validation Unobtrusive, AJAX Unobtrusive via LibMan
Packages	`SixLabors.ImageSharp` 3.2.0 (4.0+ needs a licence), `X.PagedList.Mvc.Core`
Base project	Practical 10 (WMS-10, Simple Room Reservation). Someone must first check whether it already has working login code.
IDE	Visual Studio 2026 Community, with Claude Code in the terminal and a shared `CLAUDE.md`

2. Roles and member types

- **Super Admin:** one seeded account. Only this account creates, edits, blocks or deletes admin accounts, and it cannot be edited or deleted in the UI.
- **Admin:** manages members, facilities, time slots and bookings.
- **Member:** books facilities and views own history. Every member starts as a Public member.
- **Student status:** a Public member can apply at any time from their profile with a student ID. An admin approves or rejects once, with a short reason shown to the member. After approval every booking is free. A rejected member can re-apply, because a rejection may be a human error.
- **How it is stored:** a student ID field and a status field (None, Pending, Approved, Rejected) on Member. There are no extra roles, subclasses or tables.

3. Business rules

- **Slots:** fixed one-hour time slots.
- **Capacity:** each facility has a capacity. Courts are 1 and the gym is higher (for example 30). A slot is full when its active bookings reach that capacity.
- **Advance booking:** members can book up to 7 days ahead.
- **Price:** each facility has an hourly rate, and slots marked peak add a surcharge. The premium gym simply has a higher rate. Students pay nothing and skip the payment step.
- **Payment:** a Public booking is pending until Stripe payment (test mode) succeeds. A pending booking holds its capacity for 15 minutes, and after that it stops counting. The system checks this when it loads availability, so no background job is needed.
- **Cancellation:** Public members can cancel up to 24 hours before the slot. The booking is then marked refund due, and an admin refunds outside the system and marks it refunded. Students can cancel any time.
- **QR e-ticket:** each confirmed booking shows a QR code (holding the booking ID) on the confirmation page and in the e-receipt. A small admin check-in page validates the code, reports valid, wrong date or already used, and marks the booking checked in.

4. Core modules

Seven core modules, two or three per member, grouped so each member can build and demo their slice from sample data.

#	Core module	Functions	PIC	Load
1	Security	Login, logout, registration (optional student ID), role-based access (Super Admin, Admin, Member), password hashing, access-denied page	Member 1	Heavy
2	Admin and Member Maintenance	Admin account CRUD (Super Admin only), member account CRUD, block and unblock, profile update with student status application, review of pending student applications	Member 1	Medium
3	Booking History (Member)	Own booking list and detail, cancel own booking under the cancellation rule	Member 1	Light
4	Booking and Payment	Choose facility, date and slot, capacity check, price by member type, Stripe payment for Public members, free confirmation for students, 15-minute hold, confirmation with QR	Member 2	Heavy
5	Booking Maintenance (Admin)	Booking list and detail, filter by status, status update, QR check-in page, mark refunds as paid	Member 2	Medium
6	Facility Management and Catalog	Category CRUD, facility CRUD (name, location, capacity, hourly rate, peak surcharge, photos), member-facing facility list and detail, filter by category	Member 3	Medium

#	Core module	Functions	PIC	Load
7	Time Slot and Pricing	Operating hours, time slot CRUD, peak slot marking, availability display	Member 3	Medium

5. Additional features

Twelve additional features, four per member. Difficulty is a rough estimate.

Feature	PIC	Difficulty	Starting point
Login blocking after 3 failed attempts	Member 1	Easy	Practical 7
Email verification with token link	Member 1	Medium	Practical 8
Password reset with token link	Member 1	Medium	Practical 8
Captcha (test keys first, swap for another feature if it fights back)	Member 1	Medium	Self-study
Stripe payment API (test mode)	Member 2	Hard	Self-study
E-receipt by email	Member 2	Medium	Practical 8
QR code e-ticket with admin check-in	Member 2	Medium	Self-study
Booking and revenue charts	Member 2	Medium	Self-study
Interactive slot selection on the facility detail page	Member 3	Hard	Practicals 3 and 10
Batch time slot generation for a date range	Member 3	Medium	Practical 4
AJAX search, sort and paging on facilities	Member 3	Easy	Practicals 2 and 3

Feature	PIC	Difficulty	Starting point
Multiple photos per facility with crop and resize	Member 3	Medium	Practicals 5 and 6 (ImageSharp 3.2.0)

6. Entity design for the ERD

These fields are a starting proposal. All three members agree the final entity classes together in the first session.

Entity	Key fields	Relationships
User (parent)	Email, Name, password hash, failed-attempt count and lock time	Parent of Admin and Member
Admin	Super Admin flag	Child of User
Member	Photo, student ID, student status, review note	Child of User
Category	Id, Name	One category has many facilities
Facility	Id, Name, Location, Capacity, HourlyRate, PeakSurcharge	Belongs to one category; has many photos and time slots
FacilityPhoto	Id, FileName	Belongs to one facility
TimeSlot	Id, Date, StartTime, EndTime, IsPeak	Belongs to one facility; has many bookings
Booking	Id, Price, Status, HoldExpiresAt, RefundStatus, CheckedInAt, QR token	Belongs to one member and one time slot; has one payment (Public)
Payment	Id, Amount, Stripe reference, Status, PaidAt	Belongs to one booking

The diagram is generated from the `.mdf` using the .NET Framework 4 project steps in Chapter 5. Practical 9 says an auto-generated diagram without inheritance lines is accepted.

7. Monetization models (for the short report)

Students book free, so all revenue comes from Public members and outside partners. The report needs 3-4 models with charging logic and logical revenue estimates. Only standard pricing and payment are built in the system.

Model	Who pays	Charging logic	Estimate
Standard hourly rates with peak and off-peak pricing	Public members	Facility rate, plus the surcharge on peak slots	Public bookings per month × average price per booking
Booking packages	Public members	A bundle of bookings (for example 10) at a discount, paid up front	Packages sold per month × package price
Premium facility pricing	Public members	Higher rate for premium facilities such as the Arena gym	Premium bookings per month × premium rate
Sponsorship and advertising	Sponsors	Monthly fee for logos or banners on the site	Sponsors × monthly fee

Use real TARUMT rates where you can, and state every assumption. An optional static sponsor banner on the home page makes the last model visible.

8. How the team works

- **Entity contract first:** in one session before coding, all three write the entity classes, the initial migration and the sample data (users, facilities, time slots, bookings). After that the entities are frozen, and a change needs a pull request all three approve.
- **Sample data stands in for other modules:** nobody waits on another member's module.
- **Handoffs are links:** the facility detail page (Member 3) links to the booking page (Member 2) with the slot id in the URL. The proposed route is `/Booking/Create?slotId=`; agree the real one in the first session.
- **Login first:** Member 1 merges the basic cookie login first. Until then the others test with `[Authorize]` switched off, and turn it back on before review.

- **Git:** one branch per member, merged by pull request after another member has run it. Review rotation: Member 1 reviews Member 2, Member 2 reviews Member 3, Member 3 reviews Member 1. Member 1 also does a final pass over every controller for correct access rules.
- **Claude Code:** run it in the Visual Studio terminal from the solution folder. The shared `CLAUDE.md` tells it to do only what the prompt asks and to avoid over-engineering.

9. Review and testing checklist

Every module is reviewed and tested by someone other than its author before it counts as done.

- [ ] Every form rejects empty, too long, wrong-format and duplicate input, on both client side and server side
- [ ] Every view model has validation attributes, plus a custom validator where the rules need one
- [ ] Every action has the correct `[Authorize]` or `[Authorize(Roles = "...")]`, and a logged-out or wrong-role user is blocked when typing the URL directly
- [ ] Photo uploads reject wrong types and oversized files
- [ ] Delete and update actions handle records that are already gone or in use
- [ ] Booking edge cases work: full slot, expired hold, late cancellation, student not yet approved
- [ ] The module has enough sample data to demo every page
- [ ] The author can explain every file in the module without notes

Hold a walkthrough one week before the demo where each member explains their code to the team.

10. Rubric coverage

Rubric item (marks)	How the plan covers it
Modules outline (5), team	Section 4, with a PIC for every module and the additional features marked
Entity class diagram (5), team	Section 6, with realistic relationships and the business rules reflected in the fields

Rubric item (marks)	How the plan covers it
Monetization (10), team	Section 7, with charging logic and estimates for each model
MVC architecture (5), team	Agreed folder and naming conventions, and a shared helper class
Presentation layer (5), team	One shared layout and navigation menu, agreed before modules start
Data layer (10), team	Entities agreed on day 1, validated view models for every module
Security (10), team	Manual cookie authentication, every action protected by role, final controller audit by Member 1
Core modules (30), individual	Two or three working modules each
Additional features (20), individual	Four each, including harder items (Stripe, interactive slot selection, QR check-in)

11. To do

- [ ] Someone opens the WMS-10 template and checks for working login code and a helper class
- [ ] Commit the base project to the team repository
- [ ] Hold the first session: entity classes, initial migration, sample data and the booking route
- [ ] Set up Stripe test keys and the captcha service
- [ ] Choose the email sender settings for verification, reset and receipts
- [ ] Write the short report (modules, ERD, monetization) and collect screenshots grouped by member
- [ ] Prepare the submission: cleaned zipped solution with the `.mdf`, a README with test logins for each role, and the report PDF